

LAB 2b: Depth-First-Search.

Finding the same A to H path in DFS

To find $\Rightarrow$ Since DFS visits fewer nodes will it be the better algorithm.

Code:-

graph = { } # didn't write as it's the same

~~def~~

def DFS(start, goal):

stack = [start]

v = []

while stack:

path = stack.pop()

node = path[-1]

if node == goal:

return path, v

if node not in v:

v.append(node)

for n in reversed(graph[node]):

Stack.append(path + [n])

return None, v

P, vi = DFS('A', 'H')

print("DFS path : ", P)

print("DFS visited : ", vi)

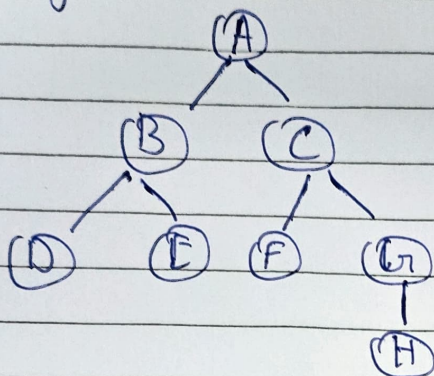
Output :-

DFS path : ['A', 'B', 'E', 'H']

DFS visited : ['A', 'B', 'D', 'E']

Answer:-

No, just cause here DFS visits fewer nodes it becomes better than BFS by using the example given



As per DFS, here ~~the~~ it would traverse A, B, D, E first and then has to back track leading to the same visit as equivalent to BFS but order difference would be A, B, D, E, C, F, G $\Rightarrow$ Here if the tree A had never ending B side branch then H would have never been found which in BFS can never happen.